

DUOC UC

INFORME TECNICO FINAL

Plataforma de Monitoreo de Calidad del Aire

Zona centro-sur de Chile

Asignatura: SCY1101 - Programacion para la Ciencia de Datos

Evaluacion Parcial Nro. 3 - Encargo con presentacion

Repositorio: <https://github.com/Cesitar16/air-quality-monitoring-platform>

Fecha: junio de 2026

Campo	Informacion
Integrantes	Cesar Rojas, Lucas Moncada, Ignacio Silva
Institucion	DUOC UC
Caso	Monitoreo calidad del aire para la zona centro-sur de Chile
Tecnologias principales	Python, FastAPI, PostgreSQL, Docker, scikit-learn, Streamlit, Plotly, pytest

Resumen ejecutivo

Este informe documenta la solución end-to-end construida para el caso de monitoreo de calidad del aire en la zona centro-sur de Chile. La plataforma integra fuentes heterogeneas, ejecuta un pipeline ETL reproducible, persiste datos en PostgreSQL, expone servicios mediante una API RESTful, aplica modelos de Machine Learning y presenta resultados mediante un dashboard interactivo diferenciado por audiencia.

La version actual del repositorio, revisada desde la rama main, incorpora ETL documentado, API FastAPI, dataset analitico, clustering K-Means, regresion supervisada para pronosticar MP2.5 a 24 horas, dashboard Streamlit, Docker Compose y pruebas automatizadas. El informe se estructura segun la pauta tecnica entregada para la evaluacion, incluyendo arquitectura, fuentes, pipeline, API, testing, Docker, modelos, dashboard, Git, conclusiones y bibliografia.

Alcance del informe

El contenido se basa en la informacion extraida del repositorio GitHub y la pauta del informe. Las capturas de pantalla reales de Swagger, Docker, dashboard, pgAdmin y resultados de pytest deben ser incorporadas desde la ejecucion local del equipo si el docente las exige como evidencia visual.

Indice

1. 1. Introduccion y contexto del caso
2. 2. Objetivos del proyecto
3. 3. Arquitectura de la solución
4. 4. Justificación de tecnologías
5. 5. Estructura del repositorio
6. 6. Pipeline ETL e integración de fuentes
7. 7. API RESTful y almacenamiento
8. 8. Seguridad, configuración y variables de entorno
9. 9. Estrategia de testing automatizado
10. 10. Containerización y despliegue con Docker
11. 11. Modelos de inteligencia: clustering y regresión
12. 12. Dashboard interactivo y visualizaciones
13. 13. Historial de cambios y colaboración Git
14. 14. Manual de ejecución del proyecto
15. 15. Conclusiones y trabajo futuro
16. 16. Bibliografía
17. 17. Anexos y checklist de evidencias

1. Introduccion y contexto del caso

El caso de negocio plantea la necesidad de una plataforma tecnologica integral para monitorear la calidad del aire en comunas de las regiones de O'Higgins, Maule, Nuble y Biobio. La motivacion principal es que actualmente los datos ambientales pueden encontrarse dispersos entre estaciones oficiales, bitacoras de fiscalizacion industrial y sensores comunitarios o IoT de bajo costo. Esta fragmentacion impide reaccionar oportunamente ante episodios de contaminacion atmosferica y dificulta entregar informacion comprensible a municipios, equipos tecnicos y vecinos.

La solucion desarrollada aborda este problema mediante una cadena completa de datos: integra fuentes, automatiza transformaciones, valida calidad, almacena en una base relacional, publica una API REST, aplica modelamiento predictivo y expone resultados a traves de un dashboard interactivo. La finalidad no es solo almacenar datos, sino transformar la informacion ambiental en indicadores, perfiles de riesgo y pronosticos que apoyen alertas tempranas.

- Los datos oficiales de estaciones se integran con sensores comunitarios, clima historico y fiscalizacion industrial.
- El sistema centraliza comunas, estaciones, industrias y mediciones en PostgreSQL.
- El equipo de ciencia de datos consume un dataset modelado para EDA, clustering y regresion.
- El dashboard adapta la informacion a usuarios ejecutivos, tecnicos y ciudadanos.

2. Objetivos del proyecto

2.1 Objetivo general

Construir una plataforma end-to-end de analitica ambiental que integre datos de calidad del aire, automatice su procesamiento, exponga servicios REST, aplique modelos de Machine Learning y entregue visualizaciones interactivas orientadas a la toma de decisiones y comunicacion ciudadana.

2.2 Objetivos especificos

- Implementar una base de datos relacional que represente comunas, estaciones, industrias y monitoreo ambiental.
- Desarrollar una API RESTful que permita consultar datos, cargar mediciones y obtener endpoints analiticos.
- Construir un pipeline ETL con lectura de multiples fuentes, transformaciones robustas, validaciones y manejo de errores.
- Generar un dataset analitico consolidado para EDA, clustering, regresion y dashboard.
- Aplicar K-Means para segmentar zonas o dias en perfiles relativos de riesgo ambiental.
- Entrenar un modelo supervisado para estimar MP2.5 esperado en las proximas 24 horas.
- Desplegar un dashboard Streamlit con vistas diferenciadas para perfil ejecutivo, tecnico y ciudadano.
- Evidenciar practicas profesionales de Git, testing, Docker y documentacion.

3. Arquitectura de la solucion

La arquitectura se organiza en capas: fuentes de datos, ETL, almacenamiento, API, ciencia de datos y visualizacion. Esta separacion permite mantener responsabilidades claras y facilita la reproducibilidad. La documentacion del repositorio define que las fuentes viven en data/raw, el modulo etl ejecuta extraccion, transformacion, validacion y carga, PostgreSQL persiste los datos, FastAPI expone

endpoints y dashboards/app.py consume artefactos ya generados para presentar las tres vistas de usuario.

Arquitectura end-to-end del sistema



Figura 1. Arquitectura general end-to-end del proyecto.

Capa	Descripcion
Fuentes	CSV/XLSX simulados de mediciones oficiales, sensores comunitarios, clima, fiscalizacion industrial y mapeo de estaciones.
ETL	Lectura, limpieza, normalizacion, validacion ambiental, carga a API y exportacion de dataset_modelado.csv.
Persistencia	PostgreSQL 16 con modelo relacional de cuatro tablas principales.
API	FastAPI + SQLAlchemy como capa de servicios y endpoints analiticos.
Modelamiento	EDA, K-Means y RandomForestRegressor usando scikit-learn.
Visualizacion	Streamlit + Plotly con vistas ejecutiva, tecnica y ciudadana.
Despliegue	Docker Compose para PostgreSQL, API y dashboard opcional.

4. Justificacion de tecnologias

Tecnologia/Libreria	Justificacion
Python	Lenguaje principal para ETL, analisis, modelos y dashboard por su ecosistema de ciencia de datos.
pandas	Manipulacion tabular, limpieza, union de fuentes,

	transformaciones y exportacion CSV.
FastAPI	Framework liviano y moderno para exponer API REST con documentacion Swagger automatica.
SQLAlchemy	Capa de acceso a datos para comunicar la API con PostgreSQL de forma mantenible.
PostgreSQL 16	Base relacional robusta para integridad, llaves foraneas, restricciones y series temporales.
scikit-learn	Entrenamiento de K-Means, regresion lineal, Random Forest, escalado, one-hot encoding y pipelines.
joblib	Serializacion de modelos y escaladores entrenados para reutilizacion posterior.
Streamlit	Construccion rapida de dashboard interactivo orientado a ciencia de datos.
Plotly	Graficos interactivos para ranking, dispersion, evolucion y distribucion de clusters.
pytest	Testing automatizado de ETL, modelamiento y artefactos.
Docker Compose	Orquestacion de servicios, base de datos, API y dashboard opcional.
PowerShell scripts	Atajos reproducibles para ejecutar ETL, clustering, regresion, dashboard y pruebas en Windows.

5. Estructura del repositorio

La estructura del repositorio fue organizada para separar pipeline de datos, API, ciencia de datos, dashboard, Docker, documentacion, pruebas y evidencias Git. Esta separacion coincide con la pauta de evaluacion y evita mezclar integracion de datos con modelamiento.

```

air-quality-monitoring-platform/
├── api/      API FastAPI, modelos y acceso a BD
├── dashboards/  app Streamlit y dependencias del dashboard
├── data/     datos raw, procesados y archivos exportados
├── db/       schema SQL y seeds de la base
├── docker/   Dockerfiles oficiales del proyecto
├── docs/     documentacion tecnica y operativa
├── etl/      extract, transform, validate, load y CLI del ETL
├── models/   artefactos persistidos de clustering y regresion
├── notebooks/  EDA, clustering y regresion narrativos
├── repo/     apoyo para evidencias Git y PRs
├── scripts/  atajos PowerShell para ejecucion local
├── src/      pipelines reutilizables de ciencia de datos
└── tests/   pruebas automatizadas

```

Decision de diseno

La carpeta etl/ se mantiene separada de src/. El ETL integra y prepara datos; src/ contiene la logica reutilizable de ciencia de datos, como clustering.py y regression.py. El dashboard no recalcula

modelos: consume artefactos ya generados.

6. Pipeline ETL e integracion de fuentes

El pipeline ETL integra fuentes simuladas de calidad del aire, normaliza los datos, aplica validaciones ambientales y prepara la carga hacia la API. El mismo flujo deja disponible `data/processed/dataset_modelado.csv` para notebooks, clustering, regresion y dashboard.

Flujo del pipeline ETL

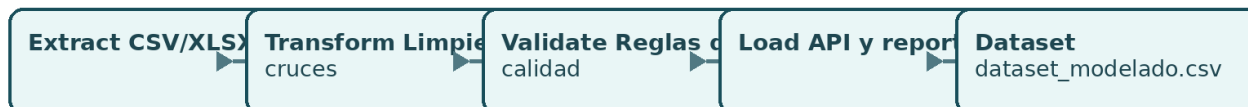


Figura 2. Flujo Extract - Transform - Validate - Load.

6.1 Fuentes de datos

Fuente	Informacion aportada
mediciones_oficiales.csv	Mediciones provenientes de estaciones oficiales del Estado.
sensores_comunitarios.csv	Lecturas de sensores comunitarios o redes ONG.
fiscalizacion_industrias.xlsx	Bitacoras de industrias, rubros y emisiones maximas permitidas.
clima_historico.csv	Variables meteorologicas: viento, direccion del viento, temperatura y humedad.
mapeo_estaciones.csv	Mapeo entre codigos raw y codigos reales de estaciones de la API.

6.2 Modulos del ETL

Modulo	Responsabilidad
etl/extract.py	Lectura de fuentes y unificacion inicial.
etl/transform.py	Limpieza, homologacion de comunas/regiones,

	conversion de tipos, cruces con clima e industrias.
etl/validate.py	Reglas de calidad, esquema minimo, contaminantes validos, humedad, direccion del viento y duplicados.
etl/load.py	Carga a API, control de omitidas, reportes y dataset analitico.
etl/run_pipeline.py	Punto de entrada CLI para modo dry-run o carga real.

6.3 Salidas del ETL

- data/processed/mediciones_limpias.csv
- data/processed/mediciones_validas.csv
- data/processed/industrias_limpias.csv
- data/processed/clima_limpio.csv
- data/processed/errores_etl.csv
- data/processed/payload_monitoreo_bulk.json
- data/processed/omitidas_carga_api.csv
- data/processed/reporte_carga_api.csv
- data/processed/dataset_modelado.csv

6.4 Validaciones y manejo de errores

El ETL valida esquema minimo obligatorio, contaminantes no negativos, humedad en rango, direccion del viento entre 0 y 360 grados, deduplicacion por estacion y fecha, y trazabilidad completa de registros invalidos. Las mediciones ya presentes en API no se reenvian, sino que se registran como omitidas. En modo --load-api, el pipeline falla explicitamente si la API no esta disponible, evitando cargas parciales no controladas.

```
# Ejecucion local del ETL
python etl/run_pipeline.py --dry-run

# Carga real contra API
python etl/run_pipeline.py --load-api

# Atajo PowerShell
.\scripts
un_etl.ps1 --load-api
```

Evidencia requerida

Insertar captura de: ejecucion del ETL, carpeta data/processed con salidas, errores_etl.csv y reporte_carga_api.csv. Estas capturas deben generarse desde el entorno local del equipo.

7. API RESTful y almacenamiento

La persistencia se implementa con PostgreSQL y una API RESTful desarrollada con FastAPI. La API actua como capa intermedia entre la base de datos y consumidores como ETL, notebooks, dashboard y Swagger.

7.1 Modelo de base de datos

La base de datos se alinea con el caso y contiene cuatro tablas principales: comunas, estaciones_sensores, industrias_fuentes y monitoreo_ambiental. El script db/schema.sql define tipos enumerados para region y tipo de sensor, llaves primarias, llaves foraneas, unicidad y restricciones de rango para contaminantes y variables meteorologicas.

Tabla	Descripcion
comunas	Municipios participantes, region, poblacion estimada e indice de vulnerabilidad respiratoria.
estaciones_sensores	Puntos de monitoreo, codigo unico, tipo de sensor, latitud, longitud y comuna.
industrias_fuentes	Fuentes fijas de contaminacion, rubro industrial, emision maxima permitida y comuna.
monitoreo_ambiental	Serie temporal con fecha/hora, estacion, MP2.5, MP10, SO2, NO2, viento, temperatura y humedad.

7.2 Endpoints de la API

Endpoint	Uso
GET /health	Verifica que la API y PostgreSQL esten operativos.
GET /comunas	Lista comunas.
GET /estaciones	Lista estaciones y sensores.
GET /industrias	Lista industrias.
GET /monitoreo	Lista mediciones con filtros y paginacion.
POST /monitoreo	Inserta una medicion individual.
POST /monitoreo/bulk	Inserta mediciones masivas, validando registros y reportando errores parciales.
GET /analytics/resumen-general	Resumen general del sistema.
GET /analytics/mp25-por-comuna	Promedio de MP2.5 por comuna.
GET /analytics/comunas-criticas	Ranking de comunas criticas.
GET /analytics/evolucion-mp25	Serie temporal de MP2.5.
GET /analytics/ica-por-comuna	Categoria ciudadana por comuna.
GET /analytics/ranking-sensores	Ranking de sensores.
GET /analytics/dataset-modelado	Dataset unido para EDA y modelamiento.

Evidencia requerida

Insertar captura de Swagger en <http://localhost:8000/docs>, prueba del endpoint /health, ejemplo de /analytics/resumen-general y captura de pgAdmin o cliente SQL mostrando tablas y datos.

8. Seguridad, configuracion y variables de entorno

La configuracion de la base y la API se externaliza mediante variables de entorno, lo que permite cambiar credenciales, host, puerto y URL de conexion sin modificar codigo fuente. Docker Compose define valores por defecto para POSTGRES_DB, POSTGRES_USER, POSTGRES_PASSWORD, POSTGRES_HOST, POSTGRES_PORT y DATABASE_URL.

```
POSTGRES_DB=calidad_aire_db
POSTGRES_USER=postgres
POSTGRES_PASSWORD=postgres
POSTGRES_HOST=db
POSTGRES_PORT=5432
DATABASE_URL=postgresql+psycopg2://postgres:postgres@db:5432/calidad_aire_db
```

- El archivo .env.example documenta las variables esperadas sin exponer credenciales reales.
- El archivo .gitignore debe impedir subir .env, entornos virtuales, caches y logs temporales.
- La API resuelve la base por el nombre del servicio db dentro de Docker Compose.
- La configuracion externa aumenta reproducibilidad y facilita despliegue local o contenedorizado.

9. Estrategia de testing automatizado

El proyecto incorpora pruebas automatizadas para validar componentes criticos. Segun la documentacion actual, la suite cubre API, ETL, clustering y regresion. En la regresion se validan columnas requeridas, correccion de caracteres, compatibilidad con datasets reducidos, construccion correcta del target a 24 horas, lags y medias moviles sin fuga temporal, entrenamiento de modelos, seleccion del mejor por RMSE, serializacion y generacion de predicciones.

Archivo de prueba	Cobertura esperada
test_etl_base.py	Validaciones generales del pipeline ETL.
test_etl_extract.py	Lectura de fuentes raw y estructura esperada.
test_etl_transform.py	Transformaciones, limpieza y cruces.
test_etl_validate.py	Reglas de calidad ambiental y trazabilidad de errores.
test_etl_load.py	Carga, omitidas, mapeo de estaciones y dataset modelado.
test_clustering.py	Preparacion, escalado, entrenamiento K-Means, etiquetas y export de artefactos.
test_regression.py	Target MP2.5 +24h, lags, entrenamiento, metricas, serializacion y predicciones.

```
# Ejecucion completa de pruebas
python -m pytest -q
```

```
# Script PowerShell
.\scripts
un_tests.ps1
```

Cobertura y evidencia

El repositorio documenta pruebas automatizadas, pero el informe no inventa porcentaje de coverage. Se debe insertar captura o salida real de pytest/coverage si se ejecuta localmente antes de la entrega.

10. Containerizacion y despliegue con Docker

La convencion vigente del proyecto define un unico docker-compose.yml en la raiz y Dockerfiles oficiales dentro de docker/. El despliegue considera tres servicios: db, api y dashboard. La base PostgreSQL se inicializa con schema y seeds al crearse el volumen; la API depende de que la base este saludable; y el dashboard se activa opcionalmente por profile.

Componente Docker	Funcion
db	PostgreSQL 16 con schema y seeds del caso.
api	FastAPI + Uvicorn, conectado a PostgreSQL.
dashboard	Streamlit opcional por profile dashboard.
volumen postgres_data	Persistencia del estado de la base entre ejecuciones.
healthcheck	Verificacion de disponibilidad de PostgreSQL mediante pg_isready.

```
# Levantar base y API
docker compose up --build

# Levantar tambien el dashboard
docker compose --profile dashboard up --build

# Reinicializar base desde cero
docker compose down -v
docker compose up --build

# Validar API
curl http://localhost:8000/health
```

Evidencia requerida

Insertar captura de docker compose up, contenedores activos, Swagger abierto en localhost:8000/docs y dashboard en localhost:8501 si se activa el profile.

11. Modelos de inteligencia

El proyecto incorpora dos componentes de Machine Learning: un modelo no supervisado para segmentar condiciones de riesgo ambiental y un modelo supervisado de regresion para estimar MP2.5

esperado a 24 horas. Ambos consumen el dataset integrado generado por el ETL y exportado desde la API analitica.

Flujo de modelamiento y visualizacion



Figura 3. Flujo de modelamiento y visualizacion.

11.1 Modelo no supervisado: clustering

El clustering resume el dataset integrado en perfiles relativos de riesgo ambiental. La salida no reemplaza una categoria regulatoria oficial, sino que sirve para segmentacion analitica y visualizacion. El pipeline usa K-Means con `n_clusters=3`, `StandardScaler` e imputacion por mediana para variables numericas.

Aspecto	Detalle
Dataset base	data/processed/dataset_modelado.csv
Algoritmo	KMeans
Numero de clusters	3
Filas procesadas	26.622
Silhouette actual	0.2893
Artefactos	clusters_riesgo_ambiental.csv, resumen_clusters.csv, kmeans_riesgo_ambiental.joblib, scaler_clustering.joblib, metricas_clustering.json

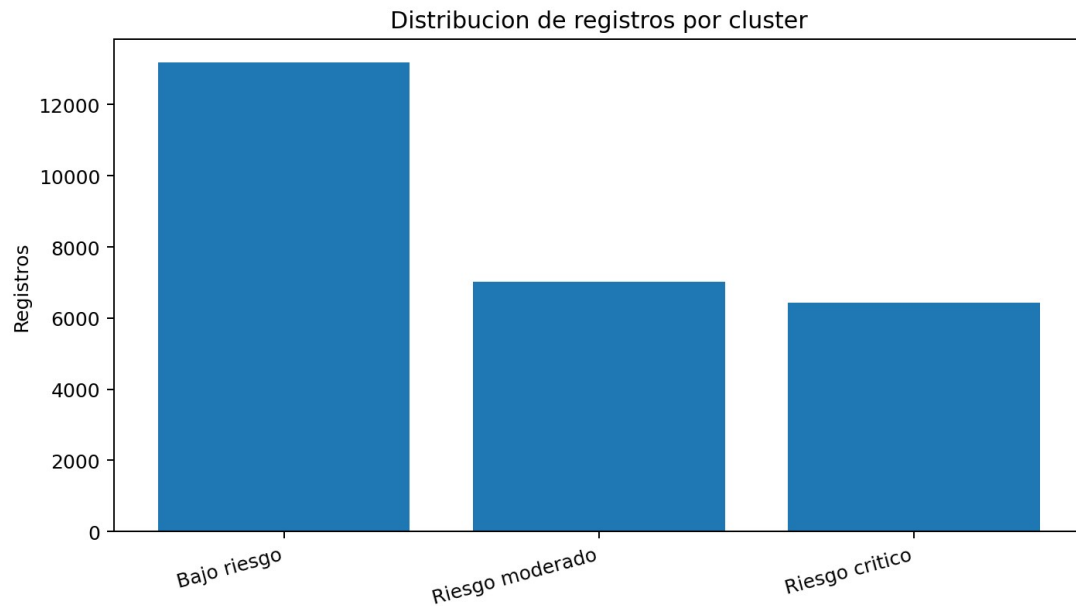


Figura 4. Distribucion de registros por cluster de riesgo ambiental.

11.2 Modelo supervisado: regresion MP2.5 +24h

La regresion responde al requerimiento del caso: estimar la concentracion esperada de Material Particulado MP2.5 para las proximas 24 horas. El modelo consume `data/processed/dataset_modelado.csv`, evita trabajar directamente sobre archivos raw y utiliza la salida oficial del ETL. Dado que la serie tiene frecuencia de 6 horas, el objetivo se construye como `mp25_24h = shift(-4)`.

Aspecto	Detalle
Variable objetivo	mp25_24h: MP2.5 futuro 24 horas despues.
Dataset final	26.622 filas en dataset_modelado.csv; 26.448 filas utiles para supervision.
Train/Test	Split temporal 80/20: 21.158 filas de entrenamiento y 5.290 de prueba.
Modelos comparados	LinearRegression y RandomForestRegressor.
Mejor modelo	RandomForestRegressor por menor RMSE.
Artefactos	modelo_mp25_24h.joblib, metricas_regresion.json, predicciones_mp25_24h.csv

11.3 Variables usadas en regresion

- Contaminacion actual: mp25, mp10, so2, no2.
- Meteorologia: velocidad_viento, direccion_viento_grados, temperatura, humedad.
- Contexto territorial: emision_maxima_permitida, indice_vulnerabilidad_respiratoria, tipo_sensor, comuna, region.
- Temporales: hora, dia, mes, dia_semana, estacion_del_ano.
- Ingenieria temporal: lag_1, lag_2, lag_3, delta_mp25, mp25_promedio_movil_3, rolling_mean_6, rolling_mean_12, direccion_viento_sin y direccion_viento_cos.

11.4 Resultados de regresion

Modelo	Metricas
LinearRegression	MAE = 3.2344 RMSE = 4.6089 R2 = 0.6736
RandomForestRegressor	MAE = 2.1487 RMSE = 3.9978 R2 = 0.7544

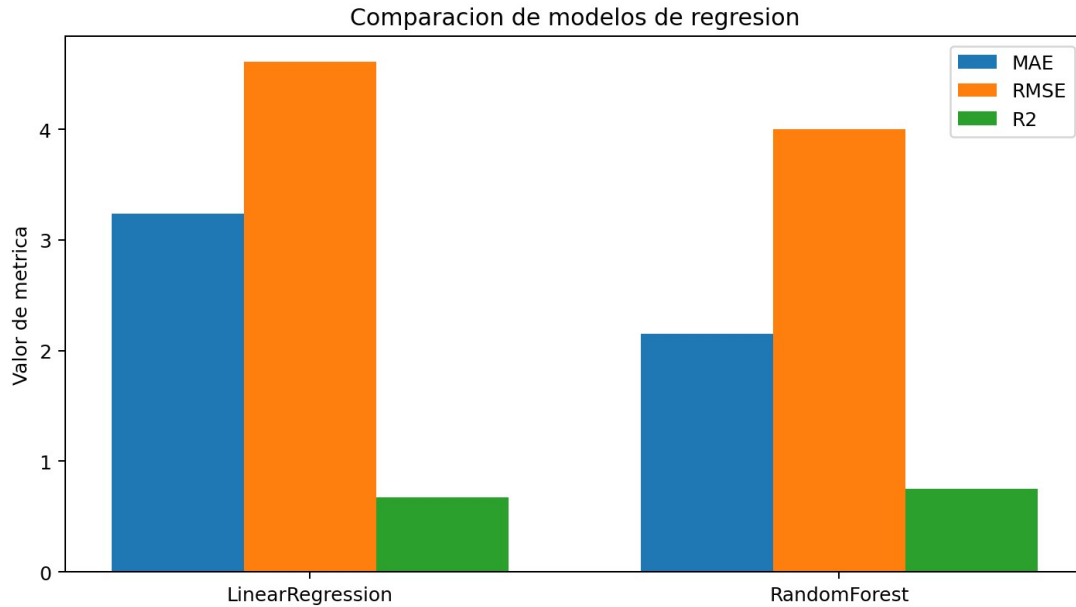


Figura 5. Comparacion de metricas de regresion.

El RandomForestRegressor fue mejor en las tres metricas evaluadas. El error absoluto medio se aproxima a 2.15 unidades de MP2.5 y el modelo explica cerca de un 75.4% de la variabilidad observada en el conjunto de prueba. Esta mejora es relevante porque el holdout final contiene 5.290 filas, lo que evita basar la evaluacion en una muestra demasiado pequena.

12. Dashboard interactivo y visualizaciones

El dashboard fue implementado con Streamlit y resume historico, clusters y pronosticos de MP2.5 en una interfaz unica. Consume artefactos ya generados, por lo que no entrena modelos ni recalcula el ETL al abrir. Esto mejora rendimiento y separa responsabilidades.

Entrada	Uso en dashboard
dataset_modelado.csv	Datos historicos e integrados.
clusters_riesgo_ambiental.csv	Registro por registro con cluster y nivel de riesgo.
resumen_clusters.csv	Resumen interpretativo de los clusters.
predicciones_mp25_24h.csv	Evaluacion y pronostico a 24 horas.
metricas_regresion.json	Metricas de los modelos comparados.
modelo_mp25_24h.joblib	Modelo entrenado para importancia de variables cuando esta disponible.

12.1 Vista Ejecutiva

- MP2.5 promedio historico.
- MP2.5 maximo.

- Comuna critica.
- Pronostico promedio a 24 horas.
- Ranking de comunas por MP2.5 predicho.

12.2 Vista Tecnica

- Metricas del modelo: MAE, RMSE y R2.
- Grafico real vs predicho.
- Importancia de variables.
- Tabla filtrable de predicciones.
- Distribucion y resumen de clusters.

12.3 Vista Ciudadana

- Seleccion de comuna.
- Estado actual.
- MP2.5 esperado a 24 horas.
- Mensaje simple de alerta.
- Detalle de estaciones consideradas.

```
# Ejecucion local del dashboard
streamlit run dashboards/app.py
```

```
# Atajo PowerShell
.\scripts
un_dashboard.ps1
```

Evidencia requerida

Insertar capturas de las tres vistas del dashboard: vista_ejecutiva.png, vista_tecnica.png y vista_ciudadana.png. La carpeta dashboards/screenshots/ esta preparada para almacenarlas.

13. Historial de cambios y colaboracion Git

El repositorio evidencia trabajo colaborativo mediante ramas funcionales y pull requests. La pauta solicita historial de ramas, merges, revisiones y evidencias de trabajo colaborativo. En la rama main se observan PRs cerrados y fusionados para ETL, EDA/clustering y regresion/dashboard.

Evidencia Git	Descripcion
PR #2 - lucas/feature/etl-pipeline	Implemento pipeline ETL end-to-end. Incluyo fuentes raw, extraccion CSV/Excel, limpieza, validacion, payload bulk, reporte de carga, dataset modelado, documentacion y tests.
PR #3 - ignacio/feature/eda-clustering	Agrego EDA y clustering de riesgo ambiental mediante K-Means. Incluyo notebooks, preprocessing, visualizations, clustering, resultados y documentacion.
PR #4 - cesar/feature/regresion-model	Completo regresion MP2.5 +24h, dashboard Streamlit, artefactos de modelos, limpieza

	Docker/docs y pruebas de regresion.
--	-------------------------------------

Integrante	Aporte principal
Lucas Moncada	ETL, fuentes, dataset procesado y documentacion del pipeline.
Ignacio Silva	EDA, clustering, perfiles de riesgo y notebooks analiticos.
Cesar Rojas	Regresion supervisada, dashboard, integracion visual, Docker/docs finales.

Evidencia requerida

Insertar capturas del historial de commits, ramas y Pull Requests en GitHub. El repositorio ya contiene PRs fusionados, pero la pauta solicita evidencia visual en el informe.

14. Manual de ejecucion del proyecto

El flujo recomendado para ejecutar el proyecto completo es instalar dependencias, levantar servicios con Docker, ejecutar ETL, clustering, regresion y dashboard.

```
# 1. Instalar dependencias
python -m pip install -r requirements.txt
```

```
# 2. Levantar base y API
docker compose up --build -d
```

```
# 3. Ejecutar ETL integrado con API
.\scripts
un_etl.ps1 --load-api
```

```
# 4. Ejecutar clustering
.\scripts
un_clustering.ps1
```

```
# 5. Ejecutar regresion
.\scripts
un_regression.ps1
```

```
# 6. Ejecutar dashboard
.\scripts
un_dashboard.ps1
```

```
# 7. Ejecutar pruebas
.\scripts
un_tests.ps1
```

14.1 Opcion con comandos Python directos

```
python etl/run_pipeline.py --load-api
python src/clustering.py
python src/regression.py
streamlit run dashboards/app.py
python -m pytest -q
```

15. Conclusiones y trabajo futuro

15.1 Conclusiones grupales

- El proyecto logro construir una solucion end-to-end coherente, desde fuentes raw hasta dashboard interactivo.
- La separacion entre ETL, API, modelamiento y visualizacion facilito la modularidad y reproducibilidad.
- El uso de PostgreSQL y FastAPI permitio centralizar datos y exponerlos a notebooks, ETL y dashboard.
- El clustering aportó una vision no supervisada de perfiles relativos de riesgo ambiental.
- La regresion MP2.5 +24h permite aproximarse a un motor de alerta temprana reutilizable por el dashboard.
- El dashboard diferencia audiencias, permitiendo comunicar resultados a niveles ejecutivos, tecnicos y ciudadanos.

15.2 Lecciones aprendidas

- Un dataset pequeno puede producir metricas poco confiables; al reexportar el dataset completo, la evaluacion de regresion mejoro y se volvio mas estable.
- Las validaciones tempranas en ETL evitan errores posteriores en API, modelos y dashboard.
- La separacion de notebooks narrativos y codigo reutilizable en src/ permite defender mejor el proyecto.
- Docker reduce diferencias de entorno y simplifica la demostracion ante el docente.
- El trabajo por ramas y PRs ayuda a repartir tareas sin bloquear al equipo.

15.3 Trabajo futuro

- Conectar fuentes reales de monitoreo ambiental o APIs publicas oficiales.
- Agregar despliegue cloud y automatizacion programada del ETL.
- Implementar alertas automaticas por correo, mensajeria o notificaciones municipales.
- Probar modelos adicionales como GradientBoostingRegressor o XGBoost.
- Evaluar generalizacion por estacion o comuna, no solo por split temporal global.
- Incorporar mapas geograficos interactivos con coordenadas de estaciones e industrias.
- Agregar monitoreo de cobertura de tests y CI/CD en GitHub Actions.

16. Bibliografia

- Repositorio del proyecto: <https://github.com/Cesitar16/air-quality-monitoring-platform>

- Documentacion oficial de FastAPI: <https://fastapi.tiangolo.com/>
- Documentacion oficial de PostgreSQL: <https://www.postgresql.org/docs/>
- Documentacion oficial de scikit-learn: <https://scikit-learn.org/stable/>
- Documentacion oficial de Streamlit: <https://docs.streamlit.io/>
- Documentacion oficial de Plotly: <https://plotly.com/python/>
- Documentacion oficial de Docker Compose: <https://docs.docker.com/compose/>
- Material de evaluacion SCY1101: pauta informe tecnico y rubrica de Evaluacion Parcial Nro. 3.
- Caso 3: Monitoreo calidad del aire para la zona centro-sur de Chile.

17. Anexos y checklist de evidencias

La pauta del informe solicita evidencia de capturas de pantalla en varias secciones. Como esas capturas deben provenir de la ejecucion real del entorno local, se deja el siguiente checklist para completar antes de la entrega final.

Seccion	Evidencia recomendada
Portada	Logo institucional si el equipo dispone del archivo oficial.
Arquitectura	Diagrama de arquitectura general del sistema.
ETL	Consola ejecutando <code>python etl/run_pipeline.py --load-api</code> y carpeta <code>data/processed</code> .
Base de datos	pgAdmin o cliente SQL mostrando tablas y datos.
API	Swagger en <code>http://localhost:8000/docs</code> y respuesta de <code>/health</code> .
Testing	Salida de <code>python -m pytest -q</code> y reporte coverage si se genera.
Docker	<code>docker compose up</code> , contenedores activos y logs sin errores.
ML	Notebook de clustering/regresion ejecutado y artefactos en <code>models/</code> y <code>data/processed/</code> .
Dashboard	Capturas de vista ejecutiva, tecnica y ciudadana.
Git	Historial de commits, PR #2, PR #3 y PR #4 fusionados.